Software Requirements Specification (SRS)

Bike Management System

---

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the complete requirements for the Bike Management System, a web-based application developed using HTML, CSS, JavaScript (Frontend) and Laravel with MySQL (Backend).
The document is intended for students, developers, project evaluators, and instructors to understand the system's functionality and design.

1.2 Scope

The Bike Management System is designed to manage bike rental operations digitally.
It allows administrators to manage bikes, users, rentals, maintenance, and payments, while customers can view bikes and book rentals online.

The system consists of:

- A static/dynamic frontend website for user interaction
- A secure backend for business logic and data management

1.3 Definitions, Acronyms, Abbreviations

Term	Description
BMS	Bike Management System
Admin	System administrator
User	Customer using the system
CRUD	Create, Read, Update, Delete
MVC	Model–View–Controller
UI	User Interface
DB	Database

---

2. Overall Description

2.1 Product Perspective

The Bike Management System is a web-based standalone application built using:

- Frontend: HTML, CSS, JavaScript
- Backend: Laravel Framework (PHP)
- Database: MySQL

The system follows the MVC architecture for scalability and maintainability.

2.2 Product Functions

The system provides the following major functionalities:

- User authentication and authorization
- Bike inventory management
- Bike booking and rental tracking
- Maintenance management
- Payment handling
- Report generation
- Role-based access control

2.3 User Classes and Characteristics

User	Description
Admin	Manages entire system
Staff	Handles maintenance and bike status
Customer	Views bikes and books rentals

2.4 Operating Environment

- Web Browser (Chrome, Edge, Firefox)
- Server: Apache (XAMPP)
- PHP 8.x
- Laravel 10+
- MySQL Database

2.5 Design Constraints

- Laravel must be used for backend
- MVC architecture mandatory
- MySQL database required
- Role-based authentication compulsory

---

3. System Features and Requirements

3.1 User Authentication

- Secure login and registration
- Password encryption
- Role-based access (Admin, Staff, Customer)

3.2 User Management

- Admin can add, edit, delete users
- Assign roles and manage user status

3.3 Bike Inventory Management

- Add, update, delete bike details
- Track bike status (Available, Rented, Maintenance)

3.4 Bike Viewing (Frontend)

- View available bikes
- Filter bikes by type and price

3.5 Bike Booking

- Book available bikes
- Select date and duration
- Cancel bookings
- Prevent double booking

3.6 Rental Management

- Track rentals and returns
- Auto-update bike availability
- Calculate rental duration

3.7 Maintenance Management

- Record bike servicing
- View maintenance history

3.8 Payment Management

- Calculate rental charges
- Store payment details
- View payment history

3.9 Dashboard & Reports

- Role-based dashboards
- Generate rental, payment, and maintenance reports

3.10 Contact & Feedback

- Contact form for users
- Admin can view messages

---

4. Frontend Requirements

4.1 General UI Requirements

- Responsive design
- White–Black–Red theme
- User-friendly navigation
- Consistent layout across pages

4.2 Frontend Pages & Features

4.2.1 Home Page

- System introduction
- Navigation menu
- Call-to-action buttons
- Feature highlights

4.2.2 Bikes Page

- Display available bikes
- Bike details (type, price, status)
- Book bike button

4.2.3 User Authentication Pages

- Login page
- Registration page
- Password validation
- Error messages

4.2.4 Dashboard (Role-Based)

- Admin dashboard
- Staff dashboard
- Customer dashboard

4.2.5 Contact Page

- Contact form
- Validation
- Message submission

---

5. Backend Requirements

5.1 Authentication & Authorization

- Secure login using Laravel Auth
- Password hashing
- Role-based access control
- Session management

5.2 User Management Module

Admin Functions:

- Create users
- Assign roles
- Update user details
- Activate / deactivate users
- Delete users

5.3 Bike Management Module

- Add bike details
- Update bike information
- Delete bikes
- **Change bike status:**
    - Available
    - Rented
    - Under Maintenance

5.4 Booking Management Module

- Create booking
- Select date & duration
- Cancel booking
- Prevent double booking
- Booking history

5.5 Rental Management Module

- Track rented bikes
- Calculate rental cost
- Update return status
- Auto-update bike availability

5.6 Maintenance Management Module

- Add maintenance record
- Schedule service
- Update maintenance status
- Maintenance history

5.7 Payment Management Module

- Calculate total rental cost
- Store payment details
- Payment history
- Payment status tracking

5.8 Notification Module

- Booking confirmation
- Status updates
- Admin alerts

5.9 Reports Module

- Rental reports
- User reports
- Payment reports
- Maintenance reports

---

6. Database Requirements

6.1 Database Tables

- users
- roles
- bikes
- bookings
- rentals
- payments

- maintenance_records
- notifications

6.2 Data Integrity

- Primary & foreign keys
- Validation rules
- Referential integrity

---

7. Non-Functional Requirements

7.1 Performance

- Handle concurrent users
- Fast page load time

7.2 Security

- CSRF protection
- Input validation
- Secure authentication
- SQL injection prevention

7.3 Usability

- Simple UI
- Easy navigation

7.4 Reliability

- High availability
- Error handling

7.5 Maintainability

- Modular Laravel code
- Easy feature upgrades

---

8. System Architecture

8.1 Architecture Overview

- MVC architecture
- RESTful routes

- Blade templates
- MySQL database

---

9. Future Enhancements

- Online payment gateway
- Mobile application
- GPS bike tracking
- Email/SMS notifications
- Admin analytics dashboard

---

10. Conclusion

The Bike Management System provides a comprehensive solution for managing bike rentals efficiently.
By integrating a modern frontend with a secure Laravel backend, the system ensures scalability, security, and ease of use, making it suitable for real-world applications and academic projects.